WEB FUNDAMENTALS

A Practical Beginner's Guide to Website Development

From Internet Basics to Building Modern Websites
with HTML, CSS, JavaScript, PHP, and MySQL

Prepared for:
Get Techy With Lucky / Tech Pulse Insider
Instructor:
Lucky Nakola

Training Roadmap: HTML | CSS | JavaScript | PHP | MySQL

Table of Contents

Table of Contents

Chapter 1: Introduction to Web Development

Welcome to your journey into web development! Before you write a single line of code, it is important to understand what web development is, why it matters, and where it fits in the bigger picture of the digital world. This chapter lays that foundation.

1.1 What Is Web Development?

Web development is the process of creating websites and web applications that run on the internet or a private network called an intranet. It covers everything from designing how a website looks, to writing the code that makes it function, to connecting it to a database that stores information.

Think of a website like a house. An architect designs the layout, a builder constructs the walls and rooms, an interior designer makes it beautiful, and a plumber or electrician adds the functional systems underneath. Web development works the same way — different roles work together to build something that is both beautiful and fully functional.

KEY NOTE: Web development is not just about writing code. It is about solving real problems for real people using technology.

1.2 Why Web Development Is Important Today

In today's world, almost every individual, business, school, organization, and government needs a digital presence. Consider these real-world examples:

- A small business in Nairobi can reach customers across Kenya and beyond through a website.
- A school in Nyeri can publish results, timetables, and announcements online.
- A freelancer can showcase their skills through a portfolio website and attract international clients.
- An NGO can run awareness campaigns, collect donations, and share reports online.
- A government agency can offer services like certificate applications or tax filing through a web portal.

The internet has become as essential as electricity. Web development is the skill that builds and maintains the infrastructure of the internet.

1.3 How Websites Help Individuals, Businesses, and Communities

Who	How Websites Help Them
Individuals	Build portfolios, sell services, blog, and establish personal brands.
Businesses	Sell products, attract clients, provide support, and manage operations online.
Schools	Share timetables, publish results, communicate with parents, and run online learning.
NGOs	Share reports, run campaigns, accept donations, and engage communities.
Governments	Offer e-services, publish policies, and communicate with citizens.

1.4 Career Opportunities in Web Development

Web development opens the door to many exciting career paths. Here are some of the most common roles:

Career Role	What They Do
Frontend Developer	Builds the visual parts of websites that users see and interact with.
Backend Developer	Handles the server, database, and application logic behind the scenes.
Full-Stack Developer	Works on both frontend and backend — a complete web developer.
UI/UX Designer	Designs how the website looks and how easy it is to use.
Web Designer	Focuses on the visual appearance, colors, fonts, and layout.
Database Administrator	Manages the databases that store website data.
DevOps Engineer	Handles website hosting, deployment, and performance.
SEO Specialist	Optimizes websites to appear higher in search engine results.
Freelance Web Developer	Works independently, building websites for various clients.

INSTRUCTOR TIP: Show learners some Kenyan tech job boards or LinkedIn listings for web developer roles in Kenya and East Africa. This makes career opportunities feel real and achievable.

1.5 Why Beginners Should Learn Fundamentals First

Many beginners are tempted to jump straight into using website builders like WordPress, Wix, or Squarespace, or to use advanced frameworks like React or Laravel. While these tools are useful, they are built ON TOP of the fundamentals.

If you do not understand the foundation, you will:

- Struggle to fix errors when things go wrong.
- Depend on tools you cannot fully control.
- Find it very hard to learn advanced concepts later.
- Be unable to build custom solutions for unique problems.

This training course follows a logical path: HTML first, then CSS, then JavaScript, then PHP, then MySQL. Each technology builds on the one before it. By the end, you will understand how the entire web development ecosystem works.

KEY NOTE: You cannot build a strong house without a strong foundation. The same is true for web development.

1.6 How Web Development Supports the Modern World

Web development is a key driver of:

- ✓ Digital Transformation: Businesses moving their operations online need web developers.
- ✓ Entrepreneurship: Developers can start web design agencies, build SaaS products, or freelance.
- ✓ Education: E-learning platforms, school portals, and educational apps are all web-based.
- ✓ Innovation: New technologies like AI, blockchain, and IoT all need web interfaces.
- ✓ Employment: The tech sector is one of the fastest-growing in Africa and globally.

1.7 Understanding the Different Types of Development

It is important to understand how web development relates to other types of software development. The table below compares four key areas:

Type	Description	Example
Website Development	Building informational or marketing websites using HTML, CSS, JS.	Portfolio site, business landing page
Web Application Development	Building interactive, data-driven apps accessed through a browser.	Gmail, Facebook, school portal
Software Development	Building desktop or mobile software installed on a device.	Microsoft Word, mobile banking app
UI/UX Design	Designing the look and feel of digital products.	Figma mockup, user flow diagram

CHAPTER SUMMARY Key takeaways from this chapter:

- Web development is the process of building websites and web applications.
- It is important for individuals, businesses, schools, NGOs, and governments.
- Career paths include frontend, backend, full-stack, and UI/UX roles.
- Learning fundamentals first gives you a strong, lasting foundation.
- Web development drives digital transformation, entrepreneurship, and innovation.
- Different types of development serve different purposes.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What is web development in your own words?
2. Name THREE career roles in web development and describe each one.
3. Why is it important to learn HTML before using tools like WordPress?
4. Give TWO examples of how a school in Kenya could benefit from having a website.
5. What is the difference between a website and a web application?
6. What does 'digital transformation' mean in the context of business?
7. Name the five technologies covered in this training program.
8. What does a UI/UX designer do?
9. How can web development support entrepreneurship in Africa?
10. List THREE ways that websites help communities.

PRACTICAL TASK: Website Review Activity

- □ Visit THREE different websites (e.g., a Kenyan business, a school, and an international company).
- □ For each website, write down:

  (a) Its purpose,

  (b) The main sections you can see,

  (c) Whether it looks professional and why.
- □ Share your findings with the class and discuss what makes each website effective or ineffective.

Chapter 2: How the Internet Works

Before you can build websites, you need to understand the environment in which they live. This chapter explains how the internet works, what happens when you type a website address, and how information travels across the world in milliseconds.

2.1 What Is the Internet?

The internet is a massive global network of computers connected together. Think of it like a giant web of roads connecting millions of cities (computers and servers) around the world. When you use your phone or laptop to open a website, you are actually sending and receiving information across this network.

The internet is not owned by any single company or government. It is a shared infrastructure that billions of people use every day. In Kenya alone, millions of people access the internet daily through mobile data, fiber connections, and Wi-Fi.

> **KEY NOTE:** The internet is the network. The World Wide Web (WWW) is a collection of websites and pages that run ON the internet. They are different things!

2.2 Clients and Servers

When you use the internet, there are always two sides to the conversation: the CLIENT and the SERVER.

Role	Description
Client	The device or browser used to REQUEST information. Your phone, laptop, or tablet is a client.
Server	A powerful computer that STORES and SENDS the website files when a client asks for them.

Every time you open a website, your browser (the client) sends a request to the server that hosts that website. The server then sends back the files (HTML, CSS, images, etc.) and your browser displays them on screen.

2.3 What Is a Web Browser?

A web browser is a software application that allows you to access and view websites. Popular browsers include Google Chrome, Mozilla Firefox, Brave, Microsoft Edge, Safari, and Opera.

The browser does several important things:

- ❖ Sends your website requests to the correct server.
- ❖ Receives the website files from the server.
- ❖ Reads and interprets HTML, CSS, and JavaScript.
- ❖ Displays the website visually on your screen.

INSTRUCTOR TIP: Ask learners to identify which browsers they use on their phones and laptops. Compare the experience of the same website on different browsers.

2.4 What Is Web Hosting?

Web hosting is a service that provides space on a server where your website files are stored. When someone wants to view your website, the server delivers those files to their browser.

Think of web hosting like renting a shop space. The building (the server) is owned by the hosting company. You rent space inside it (hosting plan) and set up your shop (your website). Examples of hosting companies used in Kenya include Safaricom, Truehost, Kenya Website Experts, and international providers like Hostinger, Bluehost, and GoDaddy.

2.5 What Is a Domain Name?

A domain name is the unique address of your website on the internet. Examples:

- google.com
- youtube.com
- kra.go.ke
- mychurchnyeri.co.ke

Domain names are purchased from companies called domain registrars. Common extensions (called TLDs - Top Level Domains) include .com, .co.ke, .org, .net, .go.ke, and .ac.ke.

2.6 What Is DNS? (The Phonebook of the Internet)

DNS stands for Domain Name System. It is the system that translates human-friendly domain names into the numerical IP addresses that computers use to find each other.

Your phone has a contacts list. You search for 'Mum' and it dials her number (e.g., +254 712 345 678). You don't need to memorize the number — the contacts list does it for you. DNS works the same way for the internet. You type 'google.com' and DNS finds the IP address (e.g., 142.250.74.78) of Google's server for you.

2.7 What Is an IP Address?

An IP address (Internet Protocol address) is a unique numerical label assigned to every device connected to the internet. It is like a street address for computers.

Example IP address: 192.168.1.1 (local) or 142.250.74.78 (Google's server)

KEY NOTE: Every server and every connected device has an IP address. Domain names are just friendly names that point to those IP addresses.

2.8 HTTP and HTTPS

Protocol	What It Means
HTTP	HyperText Transfer Protocol. The rules that govern how data is sent between a browser and a server. Data is NOT encrypted.
HTTPS	HyperText Transfer Protocol Secure. The same protocol but with encryption (SSL/TLS). Data is protected from hackers.

When you see a padlock icon in your browser's address bar, it means the website uses HTTPS and your connection is secure. Always look for HTTPS when entering passwords, credit card numbers, or personal information.

KEY NOTE: **HTTPS is not optional for modern websites. Google actually ranks HTTPS websites higher in search results. Always ensure your websites use HTTPS.**

2.9 The Full Journey: What Happens When You Open a Website?

Here is a step-by-step breakdown of what happens when you type 'google.com' into your browser:

Step	What Happens
1	You type 'google.com' into your browser and press Enter.
2	Your browser checks its local cache — has it visited this site before?
3	If not cached, the browser asks your Internet Service Provider's DNS server for the IP address of google.com.
4	The DNS server returns the IP address: e.g., 142.250.74.78.
5	Your browser sends an HTTP/HTTPS request to that IP address (Google's server).
6	Google's server receives the request and processes it.
7	The server sends back the website files: HTML, CSS, JavaScript, images.
8	Your browser reads the files and renders (displays) the Google homepage on your screen.

CHAPTER SUMMARY Key takeaways from this chapter:

- The internet is a global network of connected computers.
- Clients (browsers) request information; servers store and deliver it.
- Web hosting provides server space where your website files live.
- A domain name is your website's address (e.g., mywebsite.co.ke).
- DNS translates domain names into IP addresses — like a phonebook.
- HTTP is the protocol for transferring data; HTTPS adds security encryption.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What is the difference between the internet and the World Wide Web?
2. Define the terms 'client' and 'server' in your own words.
3. What does a web browser do?
4. What is web hosting and why do websites need it?
5. What is a domain name? Give TWO examples.
6. What does DNS stand for, and why is it called the phonebook of the internet?

7. What is an IP address? Give an example.
8. What is the difference between HTTP and HTTPS?
9. Why is HTTPS important for websites that handle user data?
10. List the 8 steps that happen when you open a website in your browser.

PRACTICAL TASK: Website Journey Diagram

- □ Draw or describe (in your own words) the full journey of opening a website.
- □ Start from: 'User types the URL' and end at: 'Website appears on screen'.
- □ Include the roles of: Browser, DNS, IP Address, Server, HTTP/HTTPS.
- □ If possible, use a free tool like draw.io to create a simple flow diagram.

Chapter 3: Frontend vs Backend Development

Every website has two sides: what you see (the frontend) and what happens behind the scenes (the backend). Understanding this distinction is one of the most important concepts in web development.

3.1 Frontend Development

Frontend development refers to everything that users see and interact with directly in their browser. It is the visual layer of a website — the design, layout, colors, buttons, menus, images, and animations.

The three core technologies of frontend development are:

Technology	Role
HTML	Provides the STRUCTURE — headings, paragraphs, images, forms, tables, links.
CSS	Provides the STYLE — colors, fonts, spacing, layout, animations, responsiveness.
JavaScript	Provides the INTERACTIVITY — button clicks, form validation, dynamic content, animations.

Examples of frontend elements:

- A navigation bar at the top of a page
- A hero section with a big background image and headline
- Buttons that change color when you hover over them
- A contact form where users type their name and email
- A slideshow of images
- A dropdown menu

3.2 Backend Development

Backend development refers to everything that happens behind the scenes — on the server. Users do not see the backend directly, but they experience its results. The backend handles data processing, user authentication, database communication, and application logic.

Examples of backend activities:

- When you log into a website, the backend checks your username and password.
- When you submit a contact form, the backend saves your message to a database and sends an email notification.
- When you make an online payment, the backend communicates with a payment gateway.
- When you view your Mpesa statement online, the backend fetches your transaction history from a database.

In this training program, we use PHP as the backend language and MySQL as the database.

KEY NOTE: You can think of frontend as the front of a restaurant — the tables, menus, and waiters. The backend is the kitchen — where the food (data) is prepared and managed.

3.3 The Database

A database is an organized collection of data stored on a server. Websites use databases to store and retrieve information such as:

- □ User accounts and passwords
- □ Blog posts and articles
- □ Products and prices (e-commerce)
- □ Student records and marks
- □ Orders and transactions

In this training, we use MySQL — one of the world's most popular relational databases. MySQL organizes data into tables with rows and columns, similar to a spreadsheet.

3.4 Frontend vs Backend vs Database Comparison

Aspect	Frontend	Backend	Database
What it does	What users see and interact with	Processes requests, runs logic	Stores and retrieves data
Technologies	HTML, CSS, JavaScript	PHP, Python, Node.js, Ruby	MySQL, PostgreSQL, MongoDB
Examples	Buttons, menus, images, forms	Login systems, payment logic	Users table, products table
Location	Runs in the browser	Runs on the server	Runs on the database server
Visibility	Fully visible to users	Hidden from users	Hidden from users

CHAPTER SUMMARY Key takeaways from this chapter:

- Frontend is what users see and interact with in their browser.
- HTML provides structure, CSS provides style, JavaScript provides interactivity.
- Backend is the server-side logic that processes requests and manages data.
- PHP is the backend language used in this training program.
- A database stores website data in organized tables.
- MySQL is the database technology used in this training.
- Frontend, backend, and database work together to make a complete website.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What is frontend development? Give THREE examples of frontend elements.
2. What is backend development? Give TWO examples of backend activities.
3. What is the role of HTML in a website?
4. What is the role of CSS in a website?
5. What is the role of JavaScript in a website?
6. What is a database and why do websites need one?
7. What is MySQL?
8. Compare frontend and backend using the restaurant analogy.
9. Which part of a website does the user NOT see directly?
10. Name TWO technologies used in backend development (other than PHP).

PRACTICAL TASK: Frontend & Backend Analysis

- □ Choose ONE website (e.g., your school's website, a bank, or a popular Kenyan website).
- □ List 5 frontend elements you can see (e.g., navigation bar, hero image, buttons).
- □ List 3 features that likely use backend/database (e.g., login, search, news updates).
- □ Write a short paragraph describing how the frontend and backend work together on that website.

Chapter 4: Website vs Web Application

One of the most common points of confusion among beginners is the difference between a website and a web application. While they share many technologies, they serve very different purposes. This chapter clears up the confusion.

4.1 What Is a Website?

A website is a collection of web pages that contain content — text, images, videos, and links — that users can read and browse. Websites are mostly informational. Their main purpose is to share information with visitors.

Examples of websites:

- ✓ A personal portfolio website showing your skills and projects
- ✓ A school website listing programs, news, and contacts
- ✓ A church website sharing sermon notes and events
- ✓ A business brochure website explaining what a company does

4.2 What Is a Web Application?

A web application (web app) is an interactive software program that runs in a browser and allows users to perform specific tasks. Unlike websites, web apps respond to user input and process data.

Examples of web applications:

- ➢ Gmail — you log in, compose emails, and manage your inbox
- ➢ Facebook — you post, comment, like, and message others
- ➢ Online banking — you check balances, transfer money, and pay bills
- ➢ A school portal — students log in to view results and download timetables
- ➢ An online shop — users browse products, add to cart, and checkout

4.3 Key Differences

Aspect	Website	Web Application
Primary Purpose	Share information	Enable users to perform tasks
User Interaction	Mostly read-only (browsing)	Highly interactive (input & output)
Login Required	Usually not required	Usually required
Database Use	Limited or none	Heavily relies on database
Examples	Portfolio, brochure site, blog	Gmail, Facebook, banking portal
Complexity	Lower	Higher

4.4 Types of Websites

Static Websites

A static website displays the same content to every visitor. The content is fixed in the HTML files and does not change unless a developer manually edits the files.

Example: A personal portfolio site with an about page, skills section, and contact information.

Dynamic Websites

A dynamic website generates content on the fly from a database. Different users may see different content based on who they are, what they search for, or what time they visit.

Example: A news website like Nation.co.ke that pulls the latest articles from a database every time a page is loaded.

Functional Websites

Websites that perform specific tasks such as processing payments, submitting forms, booking appointments, or managing accounts. These blend website and web app features.

Interactive Websites

Websites that respond to user actions with animations, live chat, quizzes, maps, or real-time updates — often using JavaScript heavily.

4.5 Summary Table: Website Types

Type	Description	Example
Static Website	Fixed content, same for all visitors	Personal portfolio, company brochure
Dynamic Website	Content loaded from database, changes over time	News site, e-commerce store
Web Application	Interactive software tasks in a browser	Gmail, school portal, online banking
Informational Website	Primarily shares information	NGO site, government website
Interactive Website	Heavy use of JS for animations/responses	Games, live chat platforms

CHAPTER SUMMARY Key takeaways from this chapter:

- A website primarily shares information; a web application enables tasks.
- Static websites have fixed content; dynamic websites load content from a database.
- Web applications require user interaction and usually need login authentication.
- Examples of web apps include Gmail, Facebook, and online banking.
- Understanding the difference helps you plan what you are building before you start coding.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What is the main difference between a website and a web application?
2. Is YouTube a website or a web application? Explain your answer.
3. What is a static website? Give one example.
4. What is a dynamic website? Give one example.
5. Why do web applications usually require users to log in?
6. Does a simple portfolio website need a database? Explain.
7. What does 'interactive website' mean?
8. Give TWO examples of web applications used in Kenya.
9. Can a website be both informational and interactive? Give an example.
10. Which type of website would you build for a church that wants to share sermon notes?

PRACTICAL TASK: Classify Online Platforms

- □ Find and classify TEN online platforms from the list below or your own choices:
- □ Platforms to consider: Google Maps, Wikipedia, KRA iTax, Jumia, Mpesa app, YouTube, a school noticeboard, Twitter/X, a government gazette, WhatsApp Web.
- □ For each platform, classify it as: Static Website, Dynamic Website, Web Application, or Mixed.
- □ Write one sentence explaining your classification for each.

Chapter 5: Types of Websites and Pages

Understanding the different categories of websites and types of web pages helps you plan better before you start building. This chapter gives you a clear map of the web development landscape.

5.1 Types of Websites

Website Type	Description
Personal Portfolio	Showcases an individual's skills, projects, and experience. Used by designers, developers, writers, and photographers.
Business Website	Represents a company online — services, team, contact info, and products.
Educational Website	Schools, colleges, and training institutions sharing programs, events, and resources.
E-Commerce Website	Online shops where users browse and purchase products. Examples: Jumia, Amazon.
Blog / News Website	Regularly updated content: articles, news, opinions, tutorials.
NGO / Community Website	Non-profit organizations sharing their mission, projects, and calls to action.
Landing Page	A single focused page designed to convert visitors into customers or subscribers.
LMS Website	Learning Management Systems for delivering online courses and training.
Web Portal	A gateway to multiple services — like a student portal or government e-services portal.

INSTRUCTOR TIP: Ask each learner to name a Kenyan example of each website type. This connects the lesson to familiar local context.

5.2 Types of Web Pages

Page Type	Purpose
Home Page	The main entry point of a website. Introduces the site and guides users to other pages.
About Page	Tells the story of the person or organization — mission, vision, team.
Services Page	Lists the services offered with descriptions, prices, and calls to action.
Contact Page	Provides ways to get in touch: form, phone, email, map, social links.
Blog Page	Displays a list of articles or posts sorted by date.
Login Page	Allows registered users to access protected areas of the site.
Dashboard Page	A protected area showing personalized data, reports, or tools.
FAQ Page	Frequently Asked Questions — helps users find answers quickly.
Privacy Policy Page	Explains how the site collects and uses user data (legally required in many countries).
Terms & Conditions Page	Legal rules and conditions for using the website.

5.3 Single Page Websites vs Multi-Page Websites

Single Page Website (SPA)

A single page website (or Single Page Application - SPA) loads one HTML page and dynamically updates the content as the user scrolls or clicks — without loading a new page. All sections (About, Services, Contact) exist on one long scrollable page.

Multi-Page Website

A multi-page website has separate HTML pages for each section. Clicking a menu link loads a new page entirely. This is the traditional website structure.

5.4 Comparison: Single Page vs Multi-Page

Aspect	Single Page Website	Multi-Page Website
Structure	One HTML page with sections	Multiple HTML pages
Navigation	Smooth scroll or anchors	Full page loads
SEO	More difficult to optimize	Easier to optimize per page
Best For	Portfolios, landing pages, simple sites	Businesses, e-commerce, blogs
Loading Speed	Fast after initial load	Each page loads separately
Complexity	Simpler to maintain	More files to manage

CHAPTER SUMMARY Key takeaways from this chapter:

- There are many types of websites: portfolio, business, e-commerce, blog, NGO, LMS, and web portals.
- Every website is made up of different types of pages: home, about, services, contact, blog, etc.
- Single page websites keep all content on one scrollable page.
- Multi-page websites use separate pages linked together through navigation.
- Knowing website types helps you plan and communicate better with clients.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. Name FIVE different types of websites and give one example of each.
2. What is the purpose of a landing page?
3. What is an LMS website? Give an example.
4. What is the difference between a Home Page and an About Page?
5. Why do websites have a Privacy Policy page?
6. What is a Single Page Website? Give one advantage.
7. What is a Multi-Page Website? Give one advantage.
8. Which type of website is better for SEO — single page or multi-page? Why?
9. What does a Dashboard page do?
10. Name THREE types of pages you would include in a school website.

PRACTICAL TASK: Plan a 5-Page Business Website

- □ You are building a website for a small Kenyan business (e.g., a salon, tech shop, or bakery).
- □ Plan the 5 pages you would create: list the page name and main content for each.
- □ Decide: will it be single-page or multi-page? Justify your choice.
- □ Write a one-paragraph description of the website's purpose and target audience.

Chapter 6: Anatomy of a Modern Website

Every modern website is made up of specific parts that work together to create a complete, professional user experience. In this chapter, we explore each major part of a website in detail, from the header at the top to the footer at the bottom.

6.1 The Header

The header is the top section of a webpage. It is usually the first thing users see and is typically visible on every page of the website. It plays a critical role in branding, navigation, and user experience.

Common Elements in a Header

- Logo — identifies the brand visually
- Navigation links — Home, About, Services, Blog, Contact
- Call-to-Action (CTA) button — e.g., 'Get a Quote', 'Book Now', 'Sign Up'
- Search icon or bar
- Mobile hamburger menu icon (for small screens)

Types of Header Behavior

Header Type	Description
Static Header	Stays at the top and scrolls away with the page content.
Sticky Header	Stays fixed at the top as you scroll down — always visible.
Fixed Header	Similar to sticky but positioned absolutely at the top of the viewport.
Transparent Header	Has a see-through background, often used over hero images.
Shrinking Header	Shrinks in height as the user scrolls down for a sleek effect.
Hamburger Menu	On mobile, navigation links collapse into a menu icon (three horizontal lines).

KEY NOTE: A sticky header is one of the most common and user-friendly choices for modern websites. It keeps navigation accessible without the user needing to scroll back to the top.

6.2 Navigation Bar

The navigation bar (navbar) is a set of links that helps users move between pages or sections of the website. Good navigation should be:

- Clear — use plain language: 'About' not 'More About Who We Are'
- Consistent — appear on every page in the same location
- Simple — limit to 5-7 main links to avoid clutter
- Responsive — work well on both desktop and mobile

For websites with many pages, dropdown menus can group related links. On mobile screens, the navbar typically collapses into a hamburger menu.

6.3 Hero Section

The hero section is the large, attention-grabbing area directly below the header. It is the first major visual impression of your website and plays a key role in capturing visitor interest within the first few seconds.

Common Hero Elements

- ✓ Headline — a short, powerful statement that communicates your value
- ✓ Subheadline — a supporting sentence with more detail
- ✓ CTA Button(s) — e.g., 'Get Started', 'Learn More', 'Contact Us'
- ✓ Background image, video, or graphic
- ✓ Optional: statistics, badges, or social proof

INSTRUCTOR TIP: Tell learners: 'You have about 3-5 seconds to capture a visitor's attention. The hero section is your one chance to make a strong first impression.'

6.4 Body / Main Content

The main body contains the primary information of the page. It is organized into sections, with each section focusing on a specific topic. Good content hierarchy means using headings, subheadings, short paragraphs, bullet points, images, and white space to make content easy to read and scan.

6.5 Common Website Sections

Section	Description
About Section	Introduces the person or organization — mission, story, team, values.
Services Section	Describes what is offered — service cards, icons, descriptions.
Features Section	Highlights the key advantages or features of a product/service.
Testimonials Section	Quotes and reviews from satisfied customers to build trust.
Pricing Section	Shows pricing plans — often with a recommended plan highlighted.
Gallery Section	Visual showcase of work, products, or events.
Blog Section	Preview of recent articles or posts.
FAQ Section	Common questions with expandable/collapsible answers (accordion style).
CTA Section	A bold section designed to encourage the visitor to take action.
Contact Section	Form, email, phone, map, and social links for getting in touch.

6.6 Contact Section

The contact section is one of the most important parts of any business or professional website. It should make it as easy as possible for visitors to reach you.

What to Include

- Contact form (name, email, message, submit button)
- Email address (clickable mailto link)
- Phone number (clickable tel link)
- WhatsApp link (important for Kenyan websites)
- Embedded Google Map showing your location
- Social media icons with links

6.7 The Footer

The footer is the bottom section of the webpage. It appears on every page and contains important supplementary information that users often look for.

Common Footer Contents

- Quick navigation links (Home, About, Services, Contact)

- Copyright notice (e.g., 2025 Lucky Nakola. All rights reserved.)
- Contact information (email, phone, address)
- Social media icons
- Newsletter signup form
- Privacy Policy and Terms & Conditions links
- Logo

KEY NOTE: Footers improve trust and usability. They give users another opportunity to navigate, find contact information, and understand the organization's legitimacy.

CHAPTER SUMMARY Key takeaways from this chapter:

- Every modern website has a header, navigation, hero, body sections, and footer.
- The header contains the logo, navigation links, and a CTA button.
- The hero section is the most impactful visual area — it creates the first impression.
- Common sections include About, Services, Testimonials, Pricing, FAQ, CTA, and Contact.
- The footer improves usability, trust, and navigation.
- Every section serves a specific purpose in guiding the user's journey.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What is a header and what does it typically contain?
2. What is a sticky header? Why is it useful?
3. What is a hamburger menu and when is it used?
4. What is the purpose of the hero section?
5. List FOUR common elements found in a hero section.
6. What should a contact section include on a Kenyan business website?
7. What is a testimonials section?
8. Why is a footer important for a professional website?
9. Name FIVE sections commonly found in a modern business website.
10. What is a CTA (Call to Action) and why is it important?

PRACTICAL TASK: Design a Homepage Layout

- □ Sketch (by hand or digitally) a simple homepage layout.
- □ Your layout must clearly show: Header (with logo and nav), Hero Section, At least 3 body sections (e.g., About, Services, Testimonials), Contact Section, Footer.
- □ Label each section clearly.
- □ Write a one-sentence description of what each section would contain.
- □ Bonus: Which section would you use as the main CTA? Why?

Chapter 7: Website Layouts and Design Structure

Layout is how the content of a webpage is arranged visually. A good layout guides users through the page in a natural and intuitive way. Before writing any code, web designers plan layouts using wireframes, mockups, and prototypes.

7.1 Common Website Layout Types

Layout Type	Description
Single Column	All content stacked vertically in one column. Common for mobile and simple blogs.
Two Column	Content split into two side-by-side columns — e.g., article + sidebar.
Grid Layout	Content arranged in a grid of rows and columns — common for portfolios and e-commerce.
Card Layout	Content displayed as individual cards, each with an image, title, and description.
Sidebar Layout	Main content on one side, a sidebar with links or ads on the other.
Dashboard Layout	Complex multi-section layout with sidebar navigation, stats, charts, and tables.
Landing Page Layout	Focused, single-purpose layout designed to drive one specific action.
Blog Layout	Featured post + grid or list of articles below.
E-Commerce Layout	Product grid with filters, search, cart, and categories.

7.2 From Idea to Prototype: The Design Process

Wireframes

A wireframe is a simple black-and-white sketch or diagram that shows the layout and structure of a webpage without any colors, images, or real content. Think of it as a blueprint of the page. Tools: pen and paper, Balsamiq, Figma, draw.io.

Mockups

A mockup is a detailed, high-fidelity design that shows what the final website will look like — with real colors, fonts, images, and branding. It is not interactive but looks like the final product. Tools: Figma, Adobe XD, Canva.

Prototypes

A prototype is an interactive model of the website. Users can click buttons and navigate between pages to test the design. It helps catch usability problems before coding begins. Tools: Figma, InVision, Adobe XD.

Aspect	Wireframe	Mockup	Prototype
Purpose	Show structure/layout	Show visual design	Test interactivity
Detail Level	Low fidelity (rough)	High fidelity (detailed)	Interactive model
Colors/Images	No	Yes	Yes
Clickable	No	No	Yes
Best For	Early planning	Client approval	User testing

7.3 Placeholder Content

During design and development, web designers use placeholder content to fill in the layout before real content is ready.

Placeholder Type	Description
Lorem Ipsum	Placeholder text that looks like Latin. Used to simulate real text without distracting from the design.
Mock Data	Fake but realistic data used during development (e.g., fake names, emails, prices).
Placeholder Images	Grey box images or royalty-free stock images used before final photos are ready.

KEY NOTE: Lorem ipsum is not real text — it is simply a visual placeholder. Always replace placeholder content with real content before launching a website.

7.4 Core Design Principles

Visual Hierarchy

Visual hierarchy guides the user's eye to the most important elements first. Large headings, bold colors, and strategic positioning all help create hierarchy.

Spacing

Good spacing (padding and margins) makes content breathable and easy to read. Crowded content overwhelms users.

Alignment

Content should be consistently aligned — left, center, or right — throughout the page for a clean, professional look.

Consistency

Use the same fonts, colors, button styles, and spacing throughout the site to maintain brand identity and professionalism.

Contrast

Use contrasting colors for text and background to ensure readability. Dark text on a light background or light text on a dark background.

Typography

Choose readable, professional fonts. Limit to 2-3 font families. Use hierarchy in font sizes: heading > subheading > body text.

Responsive Layout

The layout should adapt to different screen sizes — desktop, tablet, and mobile — so all users have a good experience.

CHAPTER SUMMARY Key takeaways from this chapter:

- Layout defines how content is arranged on a webpage.
- Common layouts include single column, grid, card, sidebar, and dashboard.
- Wireframes show structure; mockups show visual design; prototypes are interactive.
- Lorem ipsum and mock data are placeholders used during design and development.
- Core design principles include visual hierarchy, spacing, alignment, consistency, contrast, and typography.
- Responsive layout ensures the website works on all screen sizes.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What is a layout in web design?
2. Name FOUR common website layout types.
3. What is a wireframe and how is it different from a mockup?
4. What is the purpose of a prototype?
5. What is Lorem Ipsum? Why is it used?
6. What is visual hierarchy?
7. Why is consistency important in web design?
8. What does 'responsive layout' mean?
9. Which design tool is commonly used to create wireframes and mockups?
10. What is the difference between a mockup and a prototype?

PRACTICAL TASK: Create a Portfolio Wireframe

- □ Using paper, Figma, or draw.io, design a wireframe for a personal portfolio website.
- □ Your wireframe must include: Header with logo and navigation, Hero section, About section, Skills or Services section, Projects/Portfolio grid, Contact section, Footer.
- □ Label every section of your wireframe clearly.
- □ Present your wireframe to a classmate and explain your layout choices.

Chapter 8: Best Design Principles for Modern Web Developers

Building a website that looks good is only half the job. A truly great website is fast, accessible, secure, and easy to use for everyone. This chapter covers the key design principles that every modern web developer must understand and apply.

8.1 Core Design Principles

Principle	What It Means
Mobile-First Design	Design for mobile screens first, then scale up for tablets and desktops. Over 70% of internet users in Africa browse on mobile.
Responsive Design	The website automatically adjusts its layout for different screen sizes using flexible grids and CSS media queries.
Accessibility	Design for ALL users, including those with visual, hearing, or motor disabilities. Use alt text on images, good color contrast, and keyboard navigation.
Readability	Choose readable fonts, appropriate sizes, good line spacing, and sufficient contrast between text and background.
Simplicity	Avoid clutter. Every element on the page should serve a purpose. 'Less is more' in modern web design.
Consistency	Use the same colors, fonts, buttons, and spacing throughout every page of the website.
Fast Loading Speed	Optimize images, minimize code, and use caching. Slow websites lose visitors — especially on mobile data in Kenya.
Clear Call to Action	Every page should have at least one clear CTA guiding the user to take a specific action.
User-Friendly Navigation	Users should be able to find what they need in 3 clicks or fewer.
Good Color Contrast	Text must be readable against its background. Use tools like WebAIM Contrast Checker.
Clean Typography	Limit font families to 2-3. Maintain consistent heading and body text sizes.
White Space	Empty space around content is not wasted — it improves readability and elegance.
Error Handling	Forms should clearly communicate errors and guide users to correct them.

Principle	What It Means
Trust Signals	Include testimonials, certifications, privacy policy links, and HTTPS to build user confidence.
SEO-Friendly Structure	Use proper heading hierarchy (H1, H2, H3), descriptive URLs, meta tags, and alt text.
Security Awareness	Always use HTTPS, validate form inputs, and protect user data. Security starts from day one.

8.2 Why Modern Websites Should Be...

- Easy to Use: Users should accomplish their goals quickly without confusion or frustration.
- Fast: A 1-second delay in page load can reduce conversions by 7%. Speed is critical.
- Secure: Protect user data with HTTPS, input validation, and secure authentication.
- Accessible: Inclusive design ensures everyone can use your website regardless of ability.
- Scalable: Build with future growth in mind — more pages, more users, more features.
- Professional: A polished design builds trust and credibility for individuals and businesses.
- Search-Engine Friendly: SEO helps your website get found on Google and other search engines.

KEY NOTE: A website that is fast, secure, accessible, and easy to use will always outperform a beautiful website that is slow and confusing. Function first, beauty second.

INSTRUCTOR TIP: Use Google's PageSpeed Insights (pagespeed.web.dev) to analyze a popular Kenyan website and discuss the results with the class. This makes the concepts of performance and optimization very real.

CHAPTER SUMMARY Key takeaways from this chapter:

- Mobile-first design is essential — most Kenyan users browse on mobile devices.
- Responsive design ensures the website works on all screen sizes.
- Accessibility means designing for ALL users, including those with disabilities.
- Speed, security, and SEO are foundational requirements for modern websites.
- White space, clear CTAs, and consistent typography improve user experience.
- Trust signals like HTTPS, testimonials, and privacy policies build credibility.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What is mobile-first design and why is it important in Africa?
2. What is responsive design?
3. What does accessibility mean in web design?
4. Why is website loading speed important?
5. What is a Call to Action (CTA)?
6. What is white space and why is it useful?
7. Name THREE trust signals you can add to a website.
8. What is SEO? Why is it important?
9. What is the '3-click rule' in navigation design?
10. Why should all websites use HTTPS?

PRACTICAL TASK: Website Design Audit

- □ Visit a Kenyan business or school website of your choice.
- □ Evaluate it against the 16 design principles listed in this chapter.
- □ Identify 5 principles they have applied well and explain how.
- □ Identify 5 areas that need improvement and suggest how they could be improved.
- □ Write your findings in a short report of no more than one page.

Chapter 9: The Process of Making a Website

Professional web development follows a structured process. Whether you are building a simple portfolio or a complex e-commerce platform, following these steps helps ensure the project is delivered on time, within budget, and to the client's satisfaction.

9.1 The Website Development Process

Step 1: Requirement Gathering

Before touching any code or design tool, you must understand exactly what the client or user needs. Ask:

- What is the purpose of the website?
- Who is the target audience?
- What pages and features are needed?
- What content will be on the website?
- What is the budget and deadline?

Step 2: Planning

Create a clear plan before any design or code begins:

- Site Map — a visual list of all pages and how they connect
- Feature List — all functionalities required (contact form, login, gallery, etc.)
- Technology Choice — will you use plain HTML/CSS or a CMS like WordPress?
- Timeline — realistic deadlines for each stage of the project

Step 3: Design

Design the visual look and feel of the website:

- Create wireframes (low-fidelity sketches)
- Build mockups in Figma or Adobe XD
- Choose colors, fonts, and branding elements
- Get client approval on the design before coding

Step 4: Development

This is where the actual coding happens:

- HTML — structure all pages with proper semantic elements
- CSS — style the website to match the approved design
- JavaScript — add interactivity and dynamic behavior
- PHP — build backend forms, authentication, and server logic
- MySQL — create and manage the database

Step 5: Testing

Never launch a website without thorough testing:

- Browser testing — test on Chrome, Firefox, Safari, and Edge
- Mobile testing — check on different screen sizes and real devices
- Form testing — ensure all forms submit correctly and validate input
- Link testing — check that no links are broken
- Speed testing — use Google PageSpeed Insights
- Security checks — ensure forms are protected against attacks

Step 6: Deployment

Launching the website live involves:

- Purchasing a domain name (e.g., mybusiness.co.ke)
- Purchasing a hosting plan
- Configuring DNS to point the domain to the hosting server
- Uploading all website files to the server (via FTP, cPanel, or Git)
- Testing the live website

Step 7: Maintenance

A website is never truly 'done'. After launch, regular maintenance includes:

- Updating content (new blog posts, new products, news)
- Fixing bugs reported by users
- Improving SEO and performance
- Making regular backups
- Updating security patches

9.2 The Software Development Life Cycle (SDLC)

The SDLC is a formal framework for planning, creating, and maintaining software systems. It gives structure to the development process and is used by professional development teams worldwide.

Phase	What Happens
1. Planning	Define scope, resources, timeline, and budget.
2. Analysis	Gather and analyze requirements from stakeholders.
3. Design	Create wireframes, mockups, database schemas, and architecture.
4. Implementation	Write the actual code (HTML, CSS, JS, PHP, MySQL).
5. Testing	Test all features, fix bugs, and ensure quality.
6. Deployment	Launch the finished product to production (live).
7. Maintenance	Ongoing updates, bug fixes, and improvements after launch.

KEY NOTE: The SDLC is not just for large corporate projects. Even a simple student portfolio website benefits from following these 7 steps — it saves time, reduces errors, and produces better results.

CHAPTER SUMMARY Key takeaways from this chapter:

- The website development process has 7 stages: Gathering, Planning, Design, Development, Testing, Deployment, and Maintenance.
- Requirements must be clearly understood before any design or code begins.
- Testing must cover browsers, mobile, forms, links, speed, and security.
- Deployment involves domain, hosting, DNS, and uploading files.
- The SDLC provides a formal framework that mirrors the web development process.
- Maintenance is ongoing — a website always needs updates and improvements.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. Name the 7 stages of the website development process.
2. What questions should you ask during requirement gathering?
3. What is a site map?
4. Why should a client approve the design BEFORE coding begins?
5. Name FOUR types of testing you should perform before launching a website.
6. What does 'deployment' involve?
7. What is DNS configuration in the context of launching a website?
8. Why is website maintenance important after launch?
9. What does SDLC stand for?
10. How does the SDLC apply to a simple student website project?

PRACTICAL TASK: Create a Website Project Plan

- □ You have been hired to build a website for a secondary school in Kenya.
- □ Write a project plan covering: Purpose and target audience, List of all pages needed, Key features required (contact form, gallery, news, etc.), Technology choices (HTML, CSS, JS, PHP, MySQL), A simple week-by-week timeline for a 4-week project.
- □ Present your plan to the class and be ready to defend your choices.

Chapter 10: HTML Foundation

HTML — HyperText Markup Language — is the language of the web. Every webpage you have ever visited was built on HTML. It provides the structure and content of a webpage: the headings, paragraphs, images, links, lists, tables, and forms.

10.1 What Is HTML?

HTML is a markup language, not a programming language. It uses tags to define and organize content. Think of HTML as the skeleton of a webpage — it gives the body its shape and structure, but CSS adds the skin (style) and JavaScript adds the muscles (behavior).

KEY NOTE: An HTML file is simply a text file with the extension .html — you can create one in Notepad!

10.2 HTML Document Structure

Every HTML file follows a standard structure:

CODE EXAMPLE
```
<!DOCTYPE html>\n<html lang="en">\n  <head>\n    <meta charset="UTF-8">\n    <meta name="viewport" content="width=device-width, initial-scale=1.0">\n    <title>My First Website</title>\n  </head>\n  <body>\n    <h1>Hello World!</h1>\n    <p>Welcome to my first webpage.</p>\n  </body>\n</html>
```

HTML Element	Purpose
<!DOCTYPE html>	Tells the browser this is an HTML5 document.
<html>	The root element that wraps all content.
<head>	Contains metadata — title, character set, viewport, linked CSS files.
<meta charset>	Sets the character encoding to UTF-8 (supports most languages).
<title>	The text shown on the browser tab.
<body>	Contains all visible content of the webpage.

10.3 Essential HTML Tags

Tag	Purpose
<h1> to <h6>	Headings. H1 is the main title; H6 is the smallest subheading.
<p>	Paragraph of text.
<a href>	Hyperlink. Links to another page or URL.
<img src>	Displays an image.
and	Unordered list (bullets).
and	Ordered list (numbered).
<table>	Creates a table with rows and columns.
<form>	Creates a form for user input.
<input>	A form field (text, email, password, checkbox, radio, etc.).
<button>	A clickable button.
	A generic block container — used for layout and grouping.
	A generic inline container — used for styling part of text.

10.4 HTML Attributes

Attributes provide additional information about HTML elements. They are written inside the opening tag.

Examples:

- Go to Google — href specifies the URL
- <img src="photo.jpg" alt="A sunset photo"> — src specifies the image file; alt provides a description
- <input type="email" placeholder="Enter your email"> — type defines the input kind

10.5 Semantic HTML

Semantic HTML uses tags that describe the meaning of the content, not just its appearance. This is important for SEO, accessibility, and code readability.

Semantic Tag	Meaning
<header>	Defines the top section of a page or section.
<nav>	Defines the navigation links area.
<main>	Defines the primary content area of the page.

Semantic Tag	Meaning
<section>	Groups related content into a section.
<article>	Self-contained content like a blog post or news article.
<aside>	Side content (sidebar, ads, related links).
<footer>	Defines the bottom section of a page.

KEY NOTE: Using semantic tags like <header>, <nav>, <main>, and <footer> instead of generic tags makes your code cleaner, more accessible, and easier for search engines to understand.

CHAPTER SUMMARY Key takeaways from this chapter:

- HTML is the structure language of the web — every webpage uses it.
- HTML uses tags to define content: headings, paragraphs, images, links, forms.
- Every HTML document follows a standard structure with <!DOCTYPE>, <html>, <head>, and <body>.
- Attributes provide extra information about elements (href, src, alt, type).
- Semantic HTML uses meaningful tags that improve accessibility and SEO.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What does HTML stand for?
2. Is HTML a programming language? Explain.
3. What is the purpose of the <head> section in an HTML document?
4. What tag is used to create a hyperlink?
5. What attribute does an <img> tag use to specify the image file?
6. What is the difference between <ul> and ?
7. Name THREE semantic HTML tags and what they mean.
8. Why is semantic HTML important?
9. What does the 'alt' attribute do in an image tag?
10. Write the basic structure of an HTML page from memory.

PRACTICAL TASK: Build a 3-Page HTML Website

- □ Using only HTML (no CSS yet), build a simple 3-page website.
- □ Page 1: Home — Your name as H1, a short paragraph about yourself, a navigation bar with links to all 3 pages.
- □ Page 2: About — More detail about yourself, an unordered list of your interests.
- □ Page 3: Contact — A simple form with name, email, and message fields and a submit button.
- □ Use semantic tags: <header>, <nav>, <main>, <footer> on each page.
- □ Validate your HTML using validator.w3.org.

Chapter 11: CSS Foundation

CSS — Cascading Style Sheets — is the language that makes websites look beautiful. While HTML provides the structure, CSS controls the visual presentation: colors, fonts, spacing, layouts, animations, and responsiveness.

11.1 What Is CSS?

CSS is a style language that describes how HTML elements should be displayed. Without CSS, every website would look like a plain text document — black text on a white background with no visual design whatsoever.

KEY NOTE: A perfect analogy: HTML is the skeleton. CSS is the clothing and skin that makes the skeleton look presentable.

11.2 Three Ways to Apply CSS

Method	Description
Inline CSS	Written directly on the HTML element using the style attribute. Only for quick one-off changes. Example: <h1 style="color: blue;">
Internal CSS	Written inside a <style> tag in the <head> section of an HTML file. Good for single-page styling.
External CSS	Written in a separate .css file linked to the HTML. BEST PRACTICE for any real project.

11.3 CSS Selectors

CSS selectors target HTML elements to apply styles:

Selector Type	Example
Element selector	h1 { color: red; } — targets all <h1> elements
Class selector	.card { background: white; } — targets elements with class='card'
ID selector	#hero { height: 100vh; } — targets element with id='hero'
Descendant selector	nav a { color: white; } — targets <a> tags inside <nav>
Pseudo-class	a:hover { text-decoration: underline; } — applies on hover

11.4 The CSS Box Model

Every HTML element is treated as a rectangular box. The box model consists of four layers:

- Content — the actual text, image, or element
- Padding — space between the content and the border
- Border — a line around the padding
- Margin — space outside the border, separating elements

11.5 CSS Flexbox

Flexbox is a CSS layout model that makes it easy to align and distribute elements in a row or column. It is essential for building responsive navigation bars, card layouts, and hero sections.

CODE EXAMPLE .container { display: flex; justify-content: center; align-items: center; gap: 20px; }

11.6 CSS Grid

CSS Grid is a powerful two-dimensional layout system. It is ideal for creating complex page layouts with rows AND columns simultaneously.

CODE EXAMPLE .grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 24px; }

11.7 Responsive Design with Media Queries

Media queries allow different CSS rules to apply based on the screen size. This is how websites adapt to mobile, tablet, and desktop screens.

CODE EXAMPLE @media (max-width: 768px) { .grid { grid-template-columns: 1fr; } nav { flex-direction: column; } }

KEY NOTE: The most common breakpoints in responsive design: 480px (small mobile), 768px (tablet), 1024px (laptop), 1280px (desktop).

CHAPTER SUMMARY Key takeaways from this chapter:

- CSS controls the visual appearance of HTML elements.
- Three ways to apply CSS: inline, internal, and external (external is best practice).
- CSS selectors target elements by tag, class, ID, or relationship.
- The box model (content, padding, border, margin) governs element sizing and spacing.
- Flexbox handles one-dimensional layouts (row or column).
- CSS Grid handles two-dimensional layouts (rows AND columns).
- Media queries make websites responsive across different screen sizes.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What does CSS stand for?
2. Name the three ways to apply CSS to an HTML page.
3. What is the recommended (best practice) way to include CSS in a project?
4. What is a CSS class selector? Give an example.
5. List the four parts of the CSS box model.
6. What is Flexbox and when would you use it?
7. What is CSS Grid used for?
8. What is a media query?
9. Write a CSS media query that applies when the screen is 768px or smaller.
10. How does CSS turn a plain HTML page into a visually appealing website?

PRACTICAL TASK: Style the 3-Page HTML Website

- □ Take the HTML website you built in Chapter 10 and create an external CSS file.
- □ Apply the following styles: A color scheme (choose a primary and secondary color), Custom fonts using Google Fonts, Flexbox for the navigation bar, A background color or image for the hero section, Card-style layout for any list of items, Responsive design using at least one media query.
- □ The website should look professional and polished on both desktop and mobile.

Chapter 12: JavaScript Foundation

If HTML is the skeleton and CSS is the skin, then JavaScript is the muscles and brain of a webpage. JavaScript makes websites interactive, dynamic, and responsive to user actions in real time — without needing to reload the page.

12.1 What Is JavaScript?

JavaScript is a programming language that runs directly in the browser. It is the only language natively understood by all web browsers without any additional software. JavaScript can respond to user events (clicks, typing, scrolling), update page content dynamically, validate form data, and communicate with servers in the background.

KEY NOTE: JavaScript was created in just 10 days in 1995. Today, it is the most widely used programming language in the world — used for websites, mobile apps, servers, and even AI!

12.2 Core JavaScript Concepts

Variables

Variables store data that can be used and changed throughout your script.

CODE EXAMPLE let name = 'Lucky'; // String let age = 30; // Number const PI = 3.14159; // Constant (cannot be changed)

Functions

Functions are reusable blocks of code that perform a specific task.

CODE EXAMPLE function greet(name) { alert('Hello, ' + name + '!'); } greet('Lucky'); // Shows: Hello, Lucky!

Events

Events are actions that JavaScript can detect and respond to — clicks, typing, form submissions, page loading, mouse movement, etc.

CODE EXAMPLE document.getElementById('myBtn').addEventListener('click', function() { alert('Button was clicked!'); });

12.3 DOM Manipulation

The DOM (Document Object Model) is the browser's internal representation of the webpage. JavaScript can read and modify the DOM to change content, styles, or structure dynamically.

CODE EXAMPLE // Change the text content of an element document.getElementById('title').textContent = 'New Title!'; // Change the style of an element document.querySelector('.hero').style.backgroundColor = 'navy';

12.4 Form Validation

One of the most common uses of JavaScript is validating form data before it is submitted — checking that fields are not empty, emails are in the right format, passwords meet requirements, etc.

CODE EXAMPLE function validateForm() { const email = document.getElementById('email').value; if (email === '') { alert('Please enter your email address.'); return false; } return true; }

CHAPTER SUMMARY Key takeaways from this chapter:

- JavaScript adds interactivity and dynamic behavior to websites.
- Variables store data; functions perform tasks; events respond to user actions.
- The DOM allows JavaScript to read and modify the HTML structure and content.
- JavaScript can validate form data before submission to improve user experience.
- JavaScript runs in the browser — no server required for basic interactivity.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What is JavaScript and what is it used for?
2. What is the difference between 'let' and 'const'?

3. What is a function? Give an example in plain English.
4. What is an event in JavaScript? Name THREE examples of events.
5. What is the DOM?
6. How does JavaScript manipulate the DOM?
7. Why is form validation important?
8. Where does JavaScript code run — in the browser or on the server?
9. What does addEventListener() do?
10. Name TWO real-world uses of JavaScript on a modern website.

PRACTICAL TASK: Add Interactivity to Your Website

- □ Take the HTML/CSS website from the previous chapters and add JavaScript.
- □ Required features: A mobile hamburger menu that shows/hides navigation, Form validation on the contact page (check name, email, and message fields are not empty), A smooth scroll-to-top button that appears when the user scrolls down, A simple welcome alert or banner that appears when the page loads.
- □ Bonus: Add a dark/light mode toggle button.

Chapter 13: PHP Foundation

PHP (Hypertext Preprocessor) is a widely-used server-side scripting language. While JavaScript runs in the browser, PHP runs on the server. It is the engine behind millions of websites worldwide, including WordPress, Facebook (originally), and many Kenyan business platforms.

13.1 What Is PHP?

PHP is a server-side language, meaning the code runs on the web server before the result is sent to the browser. The browser never sees the PHP code — it only sees the HTML output that PHP generates.

PHP (Server-Side)	JavaScript (Client-Side)
PHP runs on the server	JavaScript runs in the browser
Processes form data	Validates form data
Connects to database	Updates page without refresh
Sends emails	Creates animations
Manages user sessions	Responds to user clicks

13.2 Basic PHP Syntax

CODE EXAMPLE <?php // Variables $name = 'Lucky'; $age = 30; // Output echo 'Hello, ' . $name . '!'; echo 'You are ' . $age . ' years old.'; ?>

Key things to remember:

- All PHP code is enclosed in <?php ... ?> tags
- Variables start with a dollar sign: $variableName
- Statements end with a semicolon: ;
- echo is used to output text to the browser

13.3 Handling HTML Forms with PHP

One of the most practical uses of PHP is processing contact forms — receiving user input and doing something with it (storing it, emailing it, etc.).

CODE EXAMPLE <?php if ($_SERVER['REQUEST_METHOD'] === 'POST') { $name = htmlspecialchars($_POST['name']); $email = htmlspecialchars($_POST['email']); $message = htmlspecialchars($_POST['message']); echo 'Thank you, ' . $name . '! We will be in touch.'; } ?>

13.4 PHP Sessions

Sessions allow PHP to remember information about a user across multiple pages — for example, keeping a user logged in as they navigate through a website.

CODE EXAMPLE <?php session_start(); $_SESSION['username'] = 'lucky_nakola'; echo 'Welcome back, ' . $_SESSION['username']; ?>

CHAPTER SUMMARY Key takeaways from this chapter:

- PHP is a server-side scripting language — it runs on the server, not in the browser.
- PHP is used for form handling, database connections, user authentication, and email sending.
- PHP variables start with $ and statements end with a semicolon.
- PHP processes form data submitted via POST or GET methods.
- Sessions allow PHP to maintain user state (e.g., login) across multiple pages.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What does PHP stand for?
2. Where does PHP code run — in the browser or on the server?
3. What symbol is used to declare a variable in PHP?
4. What is the 'echo' command used for in PHP?
5. What is the difference between PHP and JavaScript in terms of where they run?
6. How does PHP handle form data from an HTML form?
7. What is $_POST in PHP?
8. What is a PHP session used for?
9. Why is htmlspecialchars() important when processing form input?

10. Name THREE real-world tasks that PHP is used for on websites.

PRACTICAL TASK: Build a PHP Contact Form

- □ Create an HTML contact form with fields: Full Name, Email Address, Phone Number, Message.
- □ Create a PHP file (process.php) that: Receives the form data, Validates that all fields are filled, Displays a 'Thank you' message with the user's name, Optionally: saves the data to a text file or database.
- □ Test the form and make sure it handles empty submissions gracefully.

Chapter 14: MySQL Foundation

MySQL is one of the world's most popular relational database management systems. It stores website data in organized tables — users, products, orders, blog posts, and much more. When combined with PHP, MySQL enables fully dynamic, data-driven websites.

14.1 What Is a Database?

A database is an organized collection of data stored electronically on a server. Think of a database like a collection of Excel spreadsheets, but far more powerful — capable of handling millions of records, complex relationships, and secure access controls.

KEY NOTE: Without a database, a website can only show static information. With a database, it can store user accounts, process orders, publish articles, and much more.

14.2 Database Concepts

Concept	Description
Table	A collection of related data organized in rows and columns — like a spreadsheet.
Row (Record)	A single entry in a table — e.g., one user, one product, one blog post.
Column (Field)	A specific attribute of the data — e.g., name, email, price, date.
Primary Key	A unique identifier for each row — ensures no two rows are identical. Usually an auto-increment ID number.
Foreign Key	A column that links one table to another — e.g., orders linked to users.

14.3 Basic SQL Commands

SQL (Structured Query Language) is the language used to communicate with MySQL databases.

SELECT (Read Data) SELECT * FROM users WHERE city = 'Nairobi';

INSERT (Add Data) INSERT INTO users (name, email, phone) VALUES ('Lucky', 'lucky@example.com', '+254712345678');

UPDATE (Modify Data) UPDATE users SET phone = '+254799999999' WHERE id = 1;

DELETE (Remove Data) DELETE FROM users WHERE id = 5;

14.4 Creating a Table in MySQL

CREATE TABLE EXAMPLE CREATE TABLE contact_submissions (id INT AUTO_INCREMENT PRIMARY KEY, full_name VARCHAR(100) NOT NULL, email VARCHAR(150) NOT NULL, phone VARCHAR(20), message TEXT NOT NULL, submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

14.5 How PHP Connects to MySQL

PHP + MySQL CONNECTION <?php $host = 'localhost'; $user = 'root'; $pass = 'password'; $db = 'my_website'; $conn = mysqli_connect($host, $user, $pass, $db); if (!$conn) { die('Connection failed: ' . mysqli_connect_error()); } echo 'Connected successfully!'; ?>

CHAPTER SUMMARY Key takeaways from this chapter:

- MySQL is a relational database that stores website data in tables.
- Tables have rows (records) and columns (fields).
- Every table needs a Primary Key — a unique identifier for each row.
- SQL commands: SELECT (read), INSERT (add), UPDATE (modify), DELETE (remove).
- PHP connects to MySQL to create fully dynamic, database-driven websites.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What is a database?
2. What is MySQL?
3. What is a table in a database?
4. What is the difference between a row and a column?
5. What is a Primary Key?
6. What does SELECT do in SQL?
7. Write an SQL query to insert a new user into a 'users' table.
8. What does UPDATE do in SQL?
9. How does PHP connect to a MySQL database?
10. Why do websites need databases?

PRACTICAL TASK: Design a Contact Submissions Database

- □ Using MySQL (or on paper), design a table called 'contact_submissions' to store data from your PHP contact form.
- □ Include fields: id, full_name, email, phone, message, submitted_at.
- □ Write the CREATE TABLE SQL statement.
- □ Write an INSERT statement with sample data.
- □ Write a SELECT statement to retrieve all submissions where the email contains '@gmail.com'.
- □ Bonus: Connect your PHP contact form from Chapter 13 to save submissions into this MySQL table.

Chapter 15: Dashboards and Functional Website Features

A dashboard is a protected area of a website that provides a personalized, data-driven interface for logged-in users. Dashboards are found in almost every functional website: school portals, e-commerce admin panels, LMS platforms, and business management systems.

15.1 What Is a Dashboard?

A dashboard is a visual interface that displays summarized information, tools, and controls for a specific user role. Unlike the public-facing pages of a website, dashboards require login authentication and show personalized content.

Dashboard Type	Description
Admin Dashboard	Used by website administrators to manage content, users, orders, settings, and reports.
User Dashboard	Used by regular registered users to manage their own account, view history, or track activity.
Learner Dashboard	On LMS platforms — shows enrolled courses, progress, certificates, and assignments.
Business Dashboard	Sales reports, inventory levels, customer data, and financial summaries.

15.2 Common Dashboard Features

- Statistics Cards — key metrics at a glance (e.g., Total Users: 1,240 | Revenue: KES 540,000)
- Data Tables — lists of users, orders, posts, or submissions with sorting and filtering
- Charts and Graphs — visual reports (bar charts, line graphs, pie charts)
- User Management — add, edit, deactivate, or delete user accounts
- Content Management — create, edit, and publish website content
- Notifications — alerts and updates for important events
- Settings — configuration options for the website or user account

15.3 Public Pages vs Dashboard Pages

Aspect	Public Website Pages	Dashboard Pages
Access	Anyone can view	Requires login
Content	General information	Personalized data
Examples	Home, About, Services	Stats, Users, Orders
Security	No authentication	Must be authenticated

15.4 Dashboard Examples

School Portal Dashboard

A school portal dashboard might show: Student name and photo, Enrolled subjects, Exam results per subject, Timetable for the week, Fee balance, Notifications from the school.

E-Commerce Admin Dashboard

An e-commerce admin dashboard might show: Total orders today, Revenue chart for the month, Recent orders table with status (Pending, Shipped, Delivered), Low-stock product alerts, Customer messages.

CHAPTER SUMMARY Key takeaways from this chapter:

- A dashboard is a protected, data-driven interface for logged-in users.
- Admin dashboards manage the website; user dashboards show personal data.
- Common features: stats cards, data tables, charts, user management, notifications.
- Dashboard pages require authentication; public pages do not.
- Understanding dashboards helps you plan and build complete functional websites.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. What is a dashboard in web development?
2. What is the difference between an admin dashboard and a user dashboard?
3. Name FIVE common features found in a dashboard.
4. Why do dashboard pages require authentication?
5. What is a statistics card?
6. Give an example of a data table in a school portal.
7. What type of users would use a learner dashboard?
8. Why is user management an important dashboard feature?
9. How is a dashboard different from a regular public webpage?
10. Design the statistics cards you would display on a training website dashboard.

PRACTICAL TASK: Design a Training Website Dashboard

- □ On paper or Figma, sketch a dashboard layout for a training website (like this course).
- □ Include: A welcome message with the learner's name, 4 statistics cards (Courses Enrolled, Lessons Completed, Assignments Submitted, Average Score), A table showing recent course activity, A sidebar navigation (Dashboard, My Courses, Assignments, Settings, Logout).
- □ Describe what each section shows and who can see it (learner or admin).

Chapter 16: Using AI in Web Development

Artificial Intelligence (AI) tools have transformed the way web developers work. Tools like ChatGPT, GitHub Copilot, Claude, and many others can help you write code, fix errors, generate ideas, and create content faster than ever before. However, AI is a tool — not a replacement for understanding.

16.1 How AI Can Help Beginners Learn Faster

- Explain any concept in simple language — just ask!
- Generate starter code for any task — HTML templates, CSS layouts, JS functions
- Fix and debug code errors — paste your broken code and ask what is wrong
- Generate design ideas, color palettes, and layout suggestions
- Create website content: headlines, paragraphs, FAQs, About text
- Improve accessibility and SEO — ask AI to review and suggest improvements
- Create README files, project documentation, and comments

16.2 Practical AI Use Cases in Web Development

Use Case	Example Prompt
Generate HTML Structure	"Create an HTML page with a hero section, features section, and contact form."
Fix CSS Issues	"My flexbox navigation is not aligning correctly. Here is my code: [paste code]. What is wrong?"
Explain JavaScript Errors	"I get this error: 'Cannot read properties of null'. What does it mean and how do I fix it?"
Create Website Content	"Write a compelling About Us section for a web design agency in Kenya."
Improve SEO	"Review this page title and meta description and suggest improvements."
Generate Database Schema	"Design a MySQL database for an e-commerce website with users, products, and orders."

16.3 Why You Must Still Understand the Fundamentals

AI is powerful, but it has important limitations:

- AI can produce incorrect or outdated code — you need to recognize mistakes.
- AI cannot understand your specific project context the way you can.
- Copying code without understanding it means you cannot fix it when it breaks.
- Clients and employers expect you to understand what you have built.
- AI cannot replace creativity, problem-solving, and real-world communication with clients.

KEY NOTE: AI is your assistant, not your replacement. Use it to work faster, not to avoid learning. A developer who cannot code without AI is not a developer — they are a copy-paster.

16.4 Using AI Responsibly

- Always review and understand AI-generated code before using it.
- Test all AI-generated code — it can contain bugs or security vulnerabilities.
- Use AI to learn: ask it to explain the code it writes line by line.
- Do not submit AI work as your own without disclosure in academic settings.
- Keep your fundamentals strong so you can spot when AI is wrong.

INSTRUCTOR TIP: Live demonstration: Use ChatGPT or Claude to generate a simple homepage. Then ask learners to identify each HTML element, CSS property, and JavaScript function. This turns an AI tool into a learning exercise.

CHAPTER SUMMARY Key takeaways from this chapter:

- AI tools can help beginners learn faster, generate code, fix errors, and create content.
- Practical uses include generating HTML, fixing CSS, explaining errors, and writing content.
- Fundamentals remain essential — AI cannot replace true understanding.
- Always review, test, and understand AI-generated code before using it.
- Use AI as an accelerator, not as a crutch.

CHAPTER QUIZ Test your understanding by answering the following questions.

1. Name THREE AI tools that can help web developers.
2. How can AI help a beginner who is stuck on a JavaScript error?
3. Why is it dangerous to copy AI-generated code without understanding it?
4. Give TWO examples of prompts you could use with AI for web development.
5. What should you do AFTER AI generates code for you?
6. Can AI replace the need to learn HTML, CSS, and JavaScript? Explain.
7. What is one risk of relying on AI too heavily as a beginner?
8. How can AI help improve a website's SEO?
9. Name ONE task that AI can do quickly that would take a beginner a long time manually.
10. How should AI be used 'responsibly' in learning and professional environments?

PRACTICAL TASK: AI-Powered Homepage Challenge

- □ Use an AI tool (ChatGPT, Claude, or similar) to generate a complete HTML+CSS homepage.
- □ After receiving the code: Open it in a browser and screenshot the result, Go through the HTML and label each section (header, hero, about, contact, footer), Go through the CSS and explain what 5 different rules do, Identify ONE thing the AI got wrong or could improve, Make that improvement yourself manually.
- □ Write a one-paragraph reflection: 'What did I learn from using AI in this exercise?'

Chapter 17: Final Capstone Project

Congratulations on completing the core chapters of Web Fundamentals! This final capstone project is your opportunity to bring everything together — planning, designing, and building a complete, professional website using all the skills you have learned.

17.1 Project Brief

PROJECT TITLE Build a Complete Personal or Business Website

You will plan, design, and build a fully functional multi-page website. This can be a personal portfolio, a business website, or a website for a local organization or community group.

17.2 Project Requirements

Minimum Pages Required (5 pages)

- 1. Home Page — hero section, brief intro, featured services or portfolio
- 2. About Page — personal or organization story, mission, team
- 3. Services or Portfolio Page — what you offer or your work samples
- 4. Contact Page — contact form, phone, email, location
- 5. Blog or FAQ Page — at least 3 blog post previews OR 5 FAQ answers

Technical Requirements

- Semantic HTML5 structure on every page
- External CSS stylesheet with consistent styling
- Responsive design (mobile + desktop)
- Header with logo, navigation, and CTA button on all pages
- Footer with copyright and quick links on all pages
- Hero section on the home page
- Contact form with JavaScript validation
- At least 3 CSS animations or transitions
- Optional: PHP backend for contact form processing
- Optional: MySQL database to store contact form submissions

Submission Requirements

1. Source code — all HTML, CSS, JavaScript, and PHP files
2. Screenshots — at least 5 screenshots of different pages and screen sizes
3. README file — project description, technologies used, and setup instructions
4. Short reflection — 200-300 words: 'What I learned from this project'

17.3 Marking Rubric

Assessment Criterion	Max Marks	Marks Awarded
HTML Structure: Semantic tags, proper nesting, valid HTML on all pages	15	
CSS Styling: Consistent design, color scheme, typography, spacing	15	
Responsive Design: Works correctly on mobile and desktop	15	
JavaScript: Working form validation, at least 2 interactive features	15	
Content Quality: Relevant, readable, and professional content	10	
Design Quality: Visual appeal, layout, hero section, use of images	10	
Navigation: Clear, consistent navigation across all pages	5	
Footer: Present on all pages with required information	5	
Optional PHP + MySQL: Working contact form saving to database	5	
README File: Clear project description and setup instructions	3	
Reflection: Thoughtful 200-300 word reflection on learning	2	
TOTAL	**100**	

INSTRUCTOR TIP: Encourage learners to build a website for something real — their church, school, small business, or community group. A real project creates real motivation and produces portfolio-worthy work.

Glossary of Web Development Terms

This glossary contains 30 important web development terms with beginner-friendly definitions. Refer to this section whenever you encounter an unfamiliar term.

Term	Definition
Website	A collection of web pages accessible through a browser via the internet or an intranet.
Web Application	An interactive software program that runs in a browser and allows users to perform specific tasks, like Gmail or Facebook.
Frontend	The part of a website that users see and interact with directly — built with HTML, CSS, and JavaScript.
Backend	The server-side logic that runs behind the scenes — processing data, managing authentication, and communicating with databases.
Database	An organized collection of structured data stored on a server, used to store and retrieve website information.
Server	A powerful computer that stores website files and delivers them to browsers when requested.
Client	The browser or device used to access and display a website by sending requests to a server.
Browser	A software application (Chrome, Firefox, Safari) used to access and display websites.
Hosting	A service that provides server space for storing website files so they can be accessed online.
Domain Name	The unique human-friendly address of a website on the internet, e.g., mywebsite.co.ke.
DNS	Domain Name System — translates domain names into IP addresses so browsers can find the correct server.
IP Address	A unique numerical label assigned to each device on the internet, e.g., 192.168.1.1.
HTTP	HyperText Transfer Protocol — the rules governing how data is transferred between a browser and server.
HTTPS	HTTP Secure — the encrypted version of HTTP that protects data in transit using SSL/TLS.
HTML	HyperText Markup Language — the standard language for creating the structure of web pages.
CSS	Cascading Style Sheets — the language used to control the visual appearance and layout of web pages.

Term	Definition
JavaScript	A programming language that runs in the browser to add interactivity and dynamic behavior to websites.
PHP	Hypertext Preprocessor — a server-side scripting language used for backend web development and database interaction.
MySQL	A popular open-source relational database management system used to store and manage website data.
Responsive Design	A design approach where a website automatically adapts its layout to fit different screen sizes.
UI	User Interface — the visual elements users interact with: buttons, forms, menus, icons, and layout.
UX	User Experience — how users feel when using a website: ease of use, efficiency, and satisfaction.
Wireframe	A simple, low-fidelity sketch or diagram showing the basic layout and structure of a webpage.
Mockup	A detailed, high-fidelity visual design showing colors, fonts, and images — like the finished website but not clickable.
Prototype	An interactive model of a website used for testing user experience before coding begins.
Dashboard	A protected web interface that displays personalized data, reports, and management tools for logged-in users.
Static Website	A website with fixed content that does not change dynamically — the same HTML is delivered to every visitor.
Dynamic Website	A website that generates content from a database in real-time, potentially showing different content to different users.
API	Application Programming Interface — a way for different software systems to communicate and share data with each other.
SEO	Search Engine Optimization — the practice of improving a website's visibility in search engine results like Google.

About This Training Manual

This training manual — Web Fundamentals: A Practical Beginner's Guide to Website Development — was created to provide a comprehensive, accessible, and practical foundation for anyone starting their journey in web development.

Whether you are a secondary school student, a college learner, a career changer, or a small business owner who wants to understand how websites are built, this manual is designed for you.

Training Roadmap Covered

Chapter	Topic
Chapter 1	Introduction to Web Development
Chapter 2	How the Internet Works
Chapter 3	Frontend vs Backend Development
Chapter 4	Website vs Web Application
Chapter 5	Types of Websites and Pages
Chapter 6	Anatomy of a Modern Website
Chapter 7	Website Layouts and Design Structure
Chapter 8	Best Design Principles
Chapter 9	The Process of Making a Website
Chapter 10	HTML Foundation
Chapter 11	CSS Foundation
Chapter 12	JavaScript Foundation
Chapter 13	PHP Foundation
Chapter 14	MySQL Foundation
Chapter 15	Dashboards and Functional Features
Chapter 16	Using AI in Web Development
Chapter 17	Final Capstone Project

Keep building. Keep learning. Keep growing.

Get Techy With Lucky | Tech Pulse Insider

Instructor: Lucky Nakola